

Forecasting Short-Horizon CPU Thermal Events from System Telemetry

Saif Ali, Kayo Yokoyama Reis, Jacopo Francesco Calvano, Chenjie Liu

July 14, 2026

Abstract

We forecast whether CPU die temperature on Apple Silicon will reach 75 °C within the next two minutes, from telemetry sampled at 5 s. The data now holds 22 sessions and 45,328 labelled samples (16.0% positive) across two machines. Under session-blocked chronological evaluation, a tuned random forest reaches positive-class $F_1 = 0.75$ on the held-out sessions (precision 0.98, recall 0.61; ROC-AUC 0.910, PR-AUC 0.835 at 19.2% prevalence), against $F_1 = 0.57$ for a persistence baseline that fires whenever the current reading already sits at threshold. The margin over persistence is real but narrower than the absolute scores suggest, and we argue the result is now limited by the data and the label rather than by model capacity: all four model families converge to validation F_1 within 0.005 of one another, and every class-imbalance correction we tried either failed to improve on the untreated baseline or traded precision for recall at no net gain. The dominant error mode is a recall deficit (404 of 1,027 positive samples missed) concentrated, we suspect, in the early portion of approach windows. Event-level evaluation is the necessary next step before any operational claim.

1 Problem Formulation

Modern systems throttle silently when they overheat. So, we wanted to see whether a model could see it coming in advance, using nothing but the telemetry the operating system already exposes.

We set this up as binary classification. Each sample takes label $Y=1$ if CPU die temperature reaches 75 °C at any point in the following 2 min window (24 steps at 5 s sampling), and $Y=0$ otherwise. Samples whose look-ahead window is truncated by the end of a session are discarded. Treating an unobservable window as negative would bias the label toward the majority class in precisely the regions where sessions end, which is often just after a stress burst.

The horizon controls the difficulty of the task and deserves justification. Die temperature is strongly auto-correlated (Section 4), so at short horizons a rule that assumes the temperature stays where it is performs close to any learned model. We observed this at a 1 min horizon in an earlier round and lengthened it to two minutes so that the target requires anticipation rather than interpolation. Results below are specific to this horizon and this threshold.

Serial dependence also dictates the evaluation design. Positives arrive in contiguous episodes tied to workload, so row-level random splits would place parts of the same episode on both sides of the train/test boundary. We split chronologically at the session level, following the argument for blocked evaluation under dependence (Bergmeir & Benítez, 2012).

2 Main Failures and Course Corrections

Four problems shaped this project more than any modelling choice. Two of them changed the data and

the method described below.

Cross-platform telemetry, abandoned. Initially, we planned to use both macOS and Linux. We assumed that since both operating systems could log similar data, they would yield a shared dataset. But, the actual data varied wildly. Linux was missing Apple-specific metrics, and its temperature readings were stuck on a constant "nominal" value. Linux power readings also had bugs, showing spikes up to 8.6 kW. Most importantly, the Linux machine rarely got hot enough (staying below 62°C) to trigger the events we were looking for. Because of these data issues and limited resources, we decided to focus solely on macOS. As a result, we cannot guarantee our approach works on other operating systems.

Class imbalance, a negative result. The events we wanted to predict only happened about 16% of the time (and sometimes less). We tried several standard techniques to rebalance the data, but almost all of them hurt our overall accuracy (F1 score). For example, using SMOTE (a tool that generates synthetic data) failed because it blindly mixed "idle" and "stressed" computer states, creating fake data points that are physically impossible for a real computer to experience. Ultimately, the problem wasn't the imbalance itself, but how our data was grouped. Our training set happened to have a lot of stress events, while our test set had very few. No algorithm can fix that mismatch; the only real solution was going back and collecting more diverse data.

Avoiding Misleading Labels. At first, we gave the model the device's exact current temperature to help it predict the future temperature. Because temperatures change slowly, the model was essentially "cheating" by just relying on the current temperature rather than learning to predict the future. To fix this, we removed

the current temperature from the input and only gave it past temperature trends. We also set a strict baseline: if our model couldn't perform better than simply guessing "the temperature will stay exactly the same," we considered it a failure, regardless of its score.

The 1-minute horizon was initially wrong. We started by trying to predict events one minute into the future. The results looked great, but we quickly realized predicting just one minute ahead was too naive. When we extended the prediction window to two minutes, our accuracy scores dropped significantly. However, we kept the two-minute window because solving a harder, realistic problem is much better than getting artificially high scores on an unrealistic one.

3 Data and Preprocessing

The data comprises 22 sessions recorded between June 25 and July 14, 2026, on two machines (M1: 8 sessions; M5: 14), for 46,039 raw samples. Each sample records thermal pressure, CPU and GPU die temperature, per-cluster activity and frequency, CPU/GPU/ANE power, memory, swap, load averages, and battery state. Most sessions ran under a stress script alternating load bursts with cool-downs, so machines repeatedly approach and cross the threshold rather than idling.

Preprocessing trims session boundaries, forward-fills short gaps (33 gaps exceeding 7 s were detected), and removes sleeping samples. All imputation is causal: no value is filled backward. After labelling, 45,328 samples remain, 7,247 of them positive (16.0%).

Two data-quality issues carry over. CPU die temperature, from which the label is constructed, is missing in 10.9% of samples; GPU die temperature is missing in 12.7%. We impute both and retain missingness indicators. A fraction of labels therefore rests on reconstructed values, and we have still not quantified the effect. Every conclusion below is conditional on it being small, which we believe but have not shown.

4 Exploratory Findings

Figure 1 shows the marginals. Die temperature is multimodal, with an idle mode near 45 °C and a heavy upper tail reaching 93 °C. CPU power is severely right-skewed (median well below the mean of 3.59 W). Cluster frequencies concentrate at discrete DVFS states. Memory variables are bimodal because the two machines differ in footprint and we pool them. The preprocessing follows: mode-aware scaling for the multimodal columns, Box-Cox for three strongly skewed positive columns, Yeo-Johnson for signed difference features. A single global standardisation would merge the idle and stress regimes the classifier has to separate.

CPU power is the single variable most correlated with the label ($r = 0.71$), ahead of any temperature-derived quantity, and mutual information ranks it similarly. This is the physically sensible ordering: power is the forcing term and temperature the integrated re-

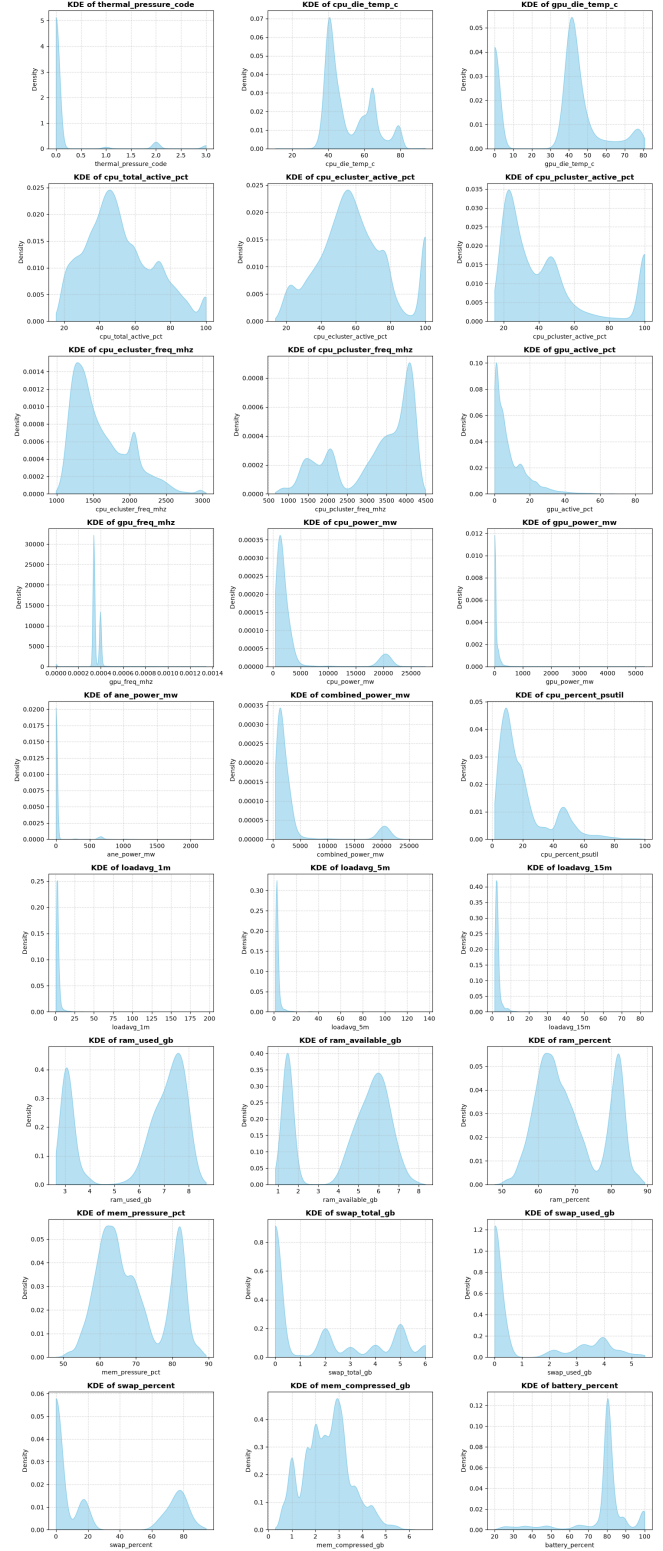


Figure 1: Marginal densities of the telemetry variables.

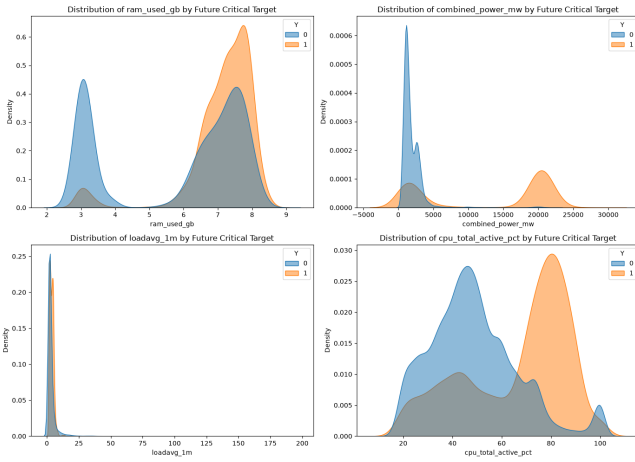


Figure 2: Class-conditional densities. Pre-event samples concentrate at higher power and activity.

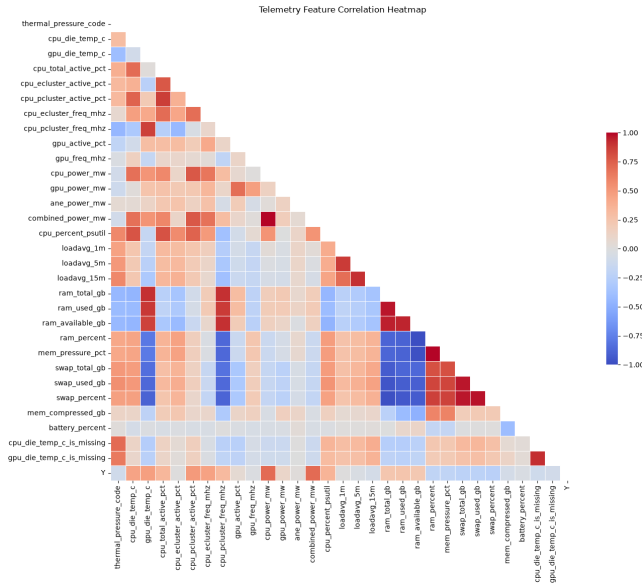


Figure 3: Pairwise correlations among telemetry variables and with the label.

sponse, so power leads. It also means the model is not simply reading the current temperature and extrapolating.

Collinearity is severe (Figure 3). Combined and CPU power are nearly identical; the three load averages track one another; the entire memory block (RAM used, available, percent, pressure, swap) is mutually correlated above $r = 0.85$, and on this data that block also correlates with P-cluster frequency and GPU die temperature. This matters for interpretation rather than prediction: coefficients of any linear model fitted here are not individually meaningful, and we do not interpret them.

The ACF decays slowly across the plotted range while the PACF cuts off after a few lags (Figure 4), consistent with a low-order autoregressive backbone plus exogenous forcing from workload. Figure 5 shows the forcing is not memoryless: identical power levels correspond to different

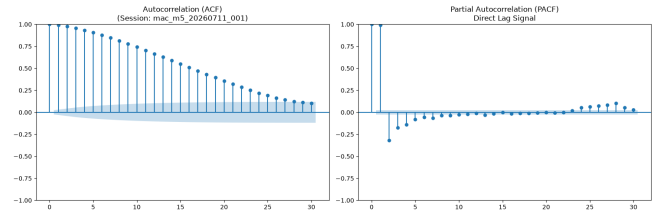


Figure 4: ACF and PACF of die temperature over 30 lags (150s).

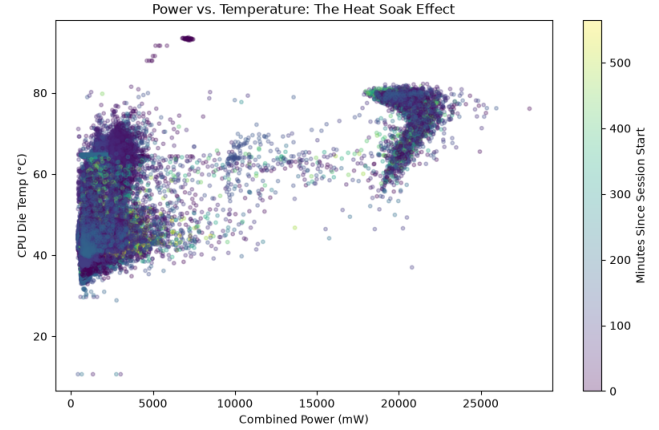


Figure 5: Die temperature against combined power, coloured by time since session start. The same power level maps to different temperatures depending on accumulated heat.

ent temperatures depending on how long the machine has been under load. This hysteresis is the empirical argument for rolling-window features over instantaneous readings, and it is borne out by the importance analysis below, where five-minute rolling statistics dominate.

5 Features

We expand five base signals (die temperature, CPU power, combined power, P-cluster activity, P-cluster frequency) into lagged values, rolling means, standard deviations, maxima, minima and ranges over windows between 30s and 15min, first differences, accelerations, and two constructed interaction terms. All features respect session boundaries; 638 boundary rows with incomplete history are dropped.

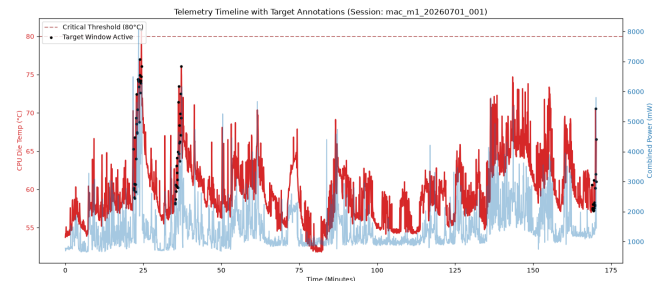


Figure 6: One stress session, with the threshold and the labelled pre-event windows marked.

Selection combined impurity importance with permutation importance on the validation split, aggregated by feature family. The two tree models disagree about which features matter, and the disagreement is informative. Random forest importance spreads across five-minute rolling statistics of power and temperature; gradient boosting concentrates 56% of its total importance on a single constructed interaction term (P-cluster power pressure). Under permutation importance on the random forest, temperature-derived features carry essentially all the positive signal (total 0.039), while the power families score *negative* (combined power -0.032 , CPU power -0.013).

We read this as collinearity, not as evidence that power is uninformative. Permutation importance is known to distribute credit arbitrarily among correlated predictors and can drive individual members negative while the group carries signal (Strobl et al., 2007). Given that CPU power and combined power are near-duplicates, this is the expected failure mode. Dropping features on the basis of individually negative permutation scores is therefore risky, and the 24-feature set we retain is better described as one plausible reduction than as an optimal subset. A grouped or conditional importance measure would settle this, and we have not run one.

6 Experimental Design

Sessions split chronologically 15/3/4 into train, validation, and test. Table 1 reports class balance. The three splits now sit within a five-point band of one another (14.3%, 17.2%, 19.2%), a marked improvement over the previous round where test prevalence was 2.9% against 15.9% in training. Prevalence is no longer the dominant obstacle, which is the main reason the numbers in Section 7 look so different from the earlier ones.

Table 1: Class balance per split.

Split	Sessions	Samples	Positives	Rate
Train	15	30,809	4,402	14.3%
Validation	3	8,524	1,465	17.2%
Test	4	5,357	1,027	19.2%

We compare logistic regression, random forests, gradient boosting, and a small feed-forward network, via scikit-learn and PyTorch. For each family we evaluated class weighting, random oversampling, SMOTE, and decision-threshold adjustment against an untreated baseline.

The imbalance sweep produced a clear and consistent pattern (Table 2). Every resampling or reweighting method traded precision for recall at a net loss in F_1 . Random forest with class weights is the sole exception, and it gains 0.016. Threshold moving is the only technique that improves on the baseline across all four families, and even then by at most 0.003. We proceeded without resampling.

Table 2: Validation F_1 by imbalance technique. Resampling reliably trades precision for recall at a net loss.

Technique	LR	RF	GB	NN
Baseline	0.787	0.782	0.789	0.789
Class weights	0.652	0.798	0.704	0.674
Random oversamp.	0.650	0.793	0.703	0.739
SMOTE	0.658	0.787	0.707	0.728
Threshold moving	0.788	0.787	0.791	0.792

SMOTE’s failure is expected rather than incidental. Interpolating between temporally distant telemetry samples synthesises machine states that violate the system’s dynamics: a point halfway between an idle sample and a stress sample corresponds to no physically realisable trajectory.

Hyperparameters were tuned by random search (five draws per family) on validation F_1 . The tuned families land within 0.005 of one another (logistic regression 0.787, random forest 0.791, gradient boosting 0.789, neural network 0.790). We selected the random forest, and we want to be explicit that a spread this small on a single validation split establishes no ordering. Four model classes with very different inductive biases, including a linear one, reaching the same performance is the strongest evidence in this report that the ceiling is set by the features and the label, not by model capacity.

7 Results

The persistence baseline predicts an event whenever the current temperature already sits at or above threshold. It reaches test $F_1 = 0.573$ (precision 0.948, recall 0.411). This is a demanding floor, and reporting it is what keeps the model’s absolute scores in perspective: much of the apparent signal in this problem is thermal inertia.

Table 3: Held-out test performance, positive class ($n = 1,027$).

	Persistence	Random forest
Precision	0.948	0.976
Recall	0.411	0.607
F_1	0.573	0.750
ROC-AUC	n/a	0.910
PR-AUC	n/a	0.835

Table 4: Test confusion matrix, random forest.

	Predicted 0	Predicted 1
Actual 0	4,315	15
Actual 1	404	623

The random forest clears the baseline by 0.18 F_1 (Table 3). PR-AUC of 0.835 at 19.2% prevalence is roughly a four-fold lift over a random ranker; we report it alongside ROC-AUC because ROC can flatter classifiers under

imbalance (Saito & Rehmsmeier, 2015).

The confusion matrix (Table 4) locates the entire deficit in recall. The model raises 638 alarms, of which 623 are correct: 15 false positives across 4,330 negative samples. It misses 404 of 1,027 positives. Whatever this model is, it is not trigger-happy; it is conservative to a fault.

Three qualifications belong next to these numbers.

First, the model’s advantage over persistence comes almost entirely from recall (0.411 to 0.607), with precision already near ceiling for both. The practical question is therefore whether an additional 20 points of recall at equal precision is worth the pipeline, and that depends on the cost of a missed event, which we have not modelled.

Second, near-perfect precision on serially dependent rows should not be read as 15 false alarms per 4,330 independent decisions. Consecutive samples within an episode share their fate; correct alarms cluster, and so do errors. The 404 misses are almost certainly not scattered uniformly. If they concentrate in the early portion of approach windows, the model is late but functional; if they span whole episodes, entire events go undetected and the row-level F_1 is flattering. The current evaluation cannot distinguish these cases, and this is the single most important gap in the report.

Third, the comparison with the previous round (test $F_1 = 0.07$ on 13 sessions) overstates modelling progress. Test prevalence rose from 2.9% to 19.2%, and the persistence baseline rose from 0.05 to 0.57 with no change to the rule whatsoever. A large share of the improvement is the evaluation regime, not the model.

8 Discussion

The defensible claim is narrow: on event-rich sessions from these two machines, telemetry features rank pre-event samples well above chance and beat thermal inertia at a two-minute horizon, with high precision and average recall. The result does not yet support claims about per-event detection, about transfer to other machines or workloads, or about the low-prevalence regime that a deployed monitor would face, where the earlier round suggests performance degrades sharply.

The convergence of four model families to within 0.005 validation F_1 , including a linear model, is the most informative observation of this round. It moves the productive margin away from model choice and toward three places. The *label*: an absolute 75 °C threshold ignores that the two machines may reach distress at different temperatures, and a per-device or normalised threshold is worth testing. The *evaluation unit*: episodes rather than rows, which changes what the numbers mean. The *operating point*: the 0.5 cutoff was inherited rather than chosen, and given a precision of 0.976 against a recall of 0.607, moving the threshold down is the obvious lever. Threshold moving already gave small gains on validation, and the selected model is plainly operating far from the

F_1 -optimal point.

We also flag an inconsistency in our own protocol. The technique sweep identified threshold moving as the best variant for three of four families, yet the final model was selected and evaluated at the default cutoff. The gap is small in F_1 terms, but it reflects a selection procedure assembled piecewise rather than specified in advance.

9 Limitations and Next Steps

Some of the known limitations are: labels partly built on imputed temperature (10.9% of the source series, effect unquantified); two machines pooled without per-device evaluation; the validation split reused for both feature selection and model selection, making validation scores optimistic by construction; feature selection performed on unscaled inputs while final models receive transformed ones; permutation importance applied to strongly collinear features, with the negative scores discussed in Section 5 not resolved; a five-draw hyperparameter search; row-level metrics under serial dependence; and four test sessions, which leaves session-level variance unquantified.

Next steps, in order of expected value:

1. Event-level evaluation. Group contiguous positives into episodes and report per-episode detection rate and lead time. This changes what the headline numbers mean and may revise them downward.
2. Threshold selection on validation, reported as a precision-recall operating curve rather than a single point. The current model is badly mis-tuned for recall.
3. Session-wise cross-validation, to attach uncertainty to the point estimates above.
4. Grouped or conditional feature importance, to resolve the negative permutation scores on the power block.
5. Per-machine evaluation and a normalised threshold in the label definition.
6. Sensitivity analysis on the imputed-temperature stretches.
7. Gather comparable data and ensure Linux compatibility, as Linux is the main OS used in production servers and is essential to our final goal. -
8. Expand the neural network model with a larger and more robust architecture.

References

- Marc Platini, Thomas Ropars, Benoit Pelletier, Noel De Palma. CPU overheating prediction in HPC systems. *29 March 2021*.
- Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191, 192–213.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.
- Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8, 25.